

Снижение размерности

PCA, t-SNE, UMAP

ML course season 2 | Паша Кашмель

Содержание

1	Зачем снижать размерность?	2
1.1	Проклятие размерности	2
1.2	Четыре мотивации для снижения размерности	2
2	PCA: метод главных компонент	2
2.1	Идея и геометрия	2
2.2	Матрица ковариации и SVD	3
2.3	Стандартизация: почему она обязательна	3
2.4	Реализация в sklearn	3
3	Выбор числа компонент	4
3.1	График объяснённой дисперсии (scree plot)	4
3.2	Trade-off качество / простота	4
4	Интерпретация PCA: loadings и биplot	5
4.1	Что такое loadings	5
4.2	Визуализация loadings	5
5	t-SNE и UMAP: нелинейная визуализация	6
5.1	Ограничения PCA	6
5.2	t-SNE	6
5.3	UMAP	6
5.4	Сравнительная таблица методов	7
5.5	Рекомендованный пайплайн для визуализации	7
6	Итоги	7

1. Зачем снижать размерность?

Реальные данные часто имеют сотни или тысячи признаков. Работать с ними напрямую трудно по нескольким причинам.

1.1. Проклятие размерности

Проклятие размерности

При увеличении числа признаков d объём пространства растёт экспоненциально. Фиксированное число обучающих примеров становится всё более разреженным. Расстояния между всеми точками выравниваются — ближайший и дальний сосед оказываются примерно одинаково далеко.

Следствие: модели, опирающиеся на расстояние (KNN, SVM с RBF-ядром, кластеризация), деградируют. Линейные модели переобучаются из-за избыточного числа параметров.

1.2. Четыре мотивации для снижения размерности

1. Визуализация. Человек способен воспринимать данные в 2D или 3D. Проекция многомерных данных в 2D позволяет «увидеть» кластеры, выбросы, разделимость классов.

2. Устранение шума. Часть дисперсии в данных объясняется шумом, а не полезным сигналом. Главные компоненты с малой дисперсией нередко соответствуют именно шуму: отбрасывая их, мы производим эффективную регуляризацию.

3. Ускорение обучения. Меньше признаков означает меньше параметров модели, меньшие матрицы, более быстрые итерации оптимизации. На задачах с тысячами признаков экономия может быть на порядки.

4. Борьба с мультиколлинеарностью. Если признаки сильно коррелируют, матрица $X^T X$ плохо обусловлена: её обращение нестабильно, коэффициенты линейной регрессии имеют большую дисперсию. PCA-компоненты по построению ортогональны, то есть их ковариация равна нулю.

2. PCA: метод главных компонент

2.1. Идея и геометрия

Главная компонента

Главная компонента (principal component) — это направление в пространстве признаков, вдоль которого дисперсия данных **максимальна**. Первая компонента — направление максимальной дисперсии. Вторая — максимальная дисперсия, ортогональная первой. И так далее.

Математически задача формулируется так: найти единичный вектор $\mathbf{w}_1$ такой, что

$$\mathbf{w}_1 = \arg \max_{\|\mathbf{w}\|=1} \text{Var}(\mathbf{w}^T X).$$

Решение — собственный вектор матрицы ковариации, отвечающий наибольшему собственному числу.

2.2. Матрица ковариации и SVD

Пусть $\tilde{X}$ — стандартизированная матрица данных (центрирована и нормирована). Матрица ковариации:

$$\Sigma = \frac{1}{n-1} \tilde{X}^\top \tilde{X} \in \mathbb{R}^{d \times d}.$$

Разложение по собственным числам: $\Sigma = V \Lambda V^\top$, где $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_d)$, $\lambda_1 \geq \lambda_2 \geq \dots \geq 0$. Столбцы V — главные компоненты (направления).

На практике `sklearn` использует SVD:

$$\tilde{X} = U S V^\top,$$

что численно устойчивее явного вычисления Σ .

2.3. Стандартизация: почему она обязательна

Стандартизация перед PCA

Если признаки имеют разные единицы измерения и масштаб (например, «рост в сантиметрах» vs. «зарплата в рублях»), PCA без стандартизации будет определяться *только* признаком с наибольшей дисперсией. Всегда применяйте `StandardScaler` до PCA.

Исключение: если все признаки имеют одинаковые единицы и смысл (например, пиксели изображения), стандартизация может быть излишней.

2.4. Реализация в sklearn

```
1  from sklearn.preprocessing import StandardScaler
2  from sklearn.decomposition import PCA
3
4  # 1. Стандартизация
5  scaler = StandardScaler()
6  X_scaled = scaler.fit_transform(X_train)
7
8  # 2. PCA
9  pca = PCA(n_components=0.95) # сохранить 95% дисперсии
10 X_pca = pca.fit_transform(X_scaled)
11
12 # Объяснённая дисперсия по компонентам
13 print(pca.explained_variance_ratio_)
14 # Суммарная
15 print(pca.explained_variance_ratio_.cumsum())
16 # Loadings
17 print(pca.components_) # shape: (n_components, n_features)
```

Параметр `n_components`

`n_components` можно задать как целое число (число компонент) или как вещественное от 0 до 1 (порог суммарной объяснённой дисперсии). `PCA(n_components=0.95)` автоматически выберет минимальное k , при котором сохраняется $\geq 95\%$ дисперсии.

3. Выбор числа компонент

3.1. График объяснённой дисперсии (scree plot)

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 pca_full = PCA()
5 pca_full.fit(X_scaled)
6
7 evr = pca_full.explained_variance_ratio_
8 cumevr = evr.cumsum()
9
10 fig, axes = plt.subplots(1, 2, figsize=(10, 4))
11
12 # Scree plot
13 axes[0].bar(range(1, len(evr)+1), evr)
14 axes[0].set_xlabel('Компонента')
15 axes[0].set_ylabel('Доля дисперсии')
16 axes[0].set_title('Scree plot')
17
18 # Cumulative
19 axes[1].plot(range(1, len(cumevr)+1), cumevr, marker='o')
20 axes[1].axhline(0.95, color='red', linestyle='--', label='95%')
21 axes[1].set_xlabel('Число компонент')
22 axes[1].set_ylabel('Накопленная дисперсия')
23 axes[1].set_title('Cumulative explained variance')
24 axes[1].legend()
25
26 plt.tight_layout()
27 plt.show()
```

3.2. Trade-off качество / простота

Выбор числа компонент зависит от задачи.

- Для визуализации: всегда $k = 2$ (или $k = 3$).
- Для препроцессинга перед моделью: начать с порога 90–95%, затем подобрать k через кросс-валидацию.

- **Метод «локтя»:** на scree-plot ищем точку, после которой прирост дисперсии становится мал — там находится «локоть».

Больше компонент не всегда лучше. Лишние компоненты могут кодировать шум, что ухудшает качество модели. Оптимальное k — то, после которого улучшение на валидации прекращается.

4. Интерпретация PCA: loadings и биplot

4.1. Что такое loadings

Каждая главная компонента — линейная комбинация исходных признаков:

$$z_k = \sum_{j=1}^d w_{jk} x_j,$$

где w_{jk} — **loading** (нагрузка) признака j на компоненту k . Хранятся в атрибуте `pca.components_` с формой `(n_components, n_features)`.

Интерпретация на примере Iris

На датасете Iris первая компонента имеет большие положительные нагрузки на `petal_length` и `petal_width`, то есть PC1 можно интерпретировать как «общий размер цветка». Вторая компонента определяется преимущественно `sepal_width` и отражает соотношение длины и ширины чашелистика.

4.2. Визуализация loadings

```
1 import pandas as pd
2 import seaborn as sns
3
4 # Heatmap нагрузок
5 loadings = pd.DataFrame(
6     pca.components_,
7     columns=feature_names,
8     index=[f'PC{i+1}' for i in range(pca.n_components_)]
9 )
10
11 plt.figure(figsize=(8, 3))
12 sns.heatmap(loadings, annot=True, cmap='coolwarm', center=0)
13 plt.title('PCA loadings')
14 plt.tight_layout()
15 plt.show()
```

5. t-SNE и UMAP: нелинейная визуализация

5.1. Ограничения PCA

PCA — линейный метод. Он хорошо работает, когда данные лежат в линейном подпространстве. Если же данные имеют нелинейную структуру (многообразие, спирали, кольца), PCA не сможет её сохранить в проекции.

5.2. t-SNE

t-SNE

t-distributed Stochastic Neighbor Embedding (Маатен, Хинтон, 2008) — метод нелинейного снижения размерности для визуализации.

Идея: строим вероятностное распределение похожести между парами точек в оригинальном пространстве (гауссово) и в 2D (распределение Стьюдента t_1), затем минимизируем KL-дивергенцию между ними.

Зачем t -распределение в 2D? Оно «тяжелохвостое» по сравнению с гауссовым: умеренно отдалённые точки отображаются ещё дальше, что визуально разделяет кластеры.

Ключевой параметр: `perplexity` (обычно 5–50). Отражает число эффективных соседей. Малое значение — тонкие локальные детали, большое — более глобальная структура.

Ограничения t-SNE

- Нет обратного преобразования (нельзя применить к новым данным).
- Расстояния *между* кластерами не несут смысла.
- Нельзя использовать проекцию как признаки для модели.
- Медленно работает при $n > 10\,000$ (квадратичная сложность).

5.3. UMAP

UMAP

Uniform Manifold Approximation and Projection (Макинни и др., 2018). Основан на топологии многообразий и теории Риманновой геометрии.

UMAP превосходит t-SNE в нескольких аспектах: он значительно быстрее, лучше сохраняет *глобальную* структуру (взаимное расположение кластеров), поддерживает метод `transform()` для новых точек.

Ключевые параметры:

- `n_neighbors`: число ближайших соседей (аналог `perplexity`, обычно 5–50).
- `min_dist`: минимальное расстояние между точками в 2D (0.0–0.5).

5.4. Сравнительная таблица методов

	PCA	t-SNE	UMAP
Тип преобразования	линейное	нелинейное	нелинейное
Скорость	быстро	медленно	быстро
<code>transform()</code> новых данных	да	нет	да
Глобальная структура	хорошо	слабо	хорошо
Интерпретируемость	высокая	низкая	низкая
Для признаков модели	да	нет	осторожно

5.5. Рекомендованный пайплайн для визуализации

```

1  from sklearn.preprocessing import StandardScaler
2  from sklearn.decomposition import PCA
3  from sklearn.manifold import TSNE
4  import umap
5
6  # 1. Стандартизация
7  X_scaled = StandardScaler().fit_transform(X)
8
9  # 2. Предварительный PCA (опционально, ускоряет t-SNE)
10 X_pca50 = PCA(n_components=50,
11               ↪ random_state=42).fit_transform(X_scaled)
12
13 # 3. t-SNE
14 X_tsne = TSNE(n_components=2, perplexity=30,
15               ↪ random_state=42).fit_transform(X_pca50)
16
17 # 4. UMAP
18 X_umap = umap.UMAP(n_components=2, n_neighbors=15,
19                    ↪ min_dist=0.1,
20                    ↪ random_state=42).fit_transform(X_scaled)

```

6. Итоги

Главные выводы

1. Снижение размерности решает проблемы визуализации, шума, скорости и мультиколлинеарности.
2. PCA — линейный метод, основанный на собственных векторах матрицы ковариации. Всегда стандартизируйте перед PCA.
3. Число компонент выбирается по графику накопленной дисперсии или через кросс-валидацию.
4. Loadings показывают, какие признаки определяют каждую компоненту.
5. t-SNE и UMAP — нелинейные методы только для визуализации. Они

не предназначены для подготовки признаков к моделям.
6. Стандартный пайплайн: `StandardScaler` → `PCA` → `t-SNE/UMAP`.

Полезные ресурсы: [Яндекс ML Handbook](#) · [sklearn: decomposition](#) · [umap-learn docs](#)